

A Dynamically Reconfigurable Systolic Array Architecture for Parallel Matrix Multiplication

Abstract—Systolic arrays (SAs) are widely used to accelerate matrix computations owing to their regular structure and high spatial parallelism. Conventional fixed-dimension designs suffer from poor hardware utilisation when processing workloads smaller than the array, a limitation that has grown acute in large language model (LLM) inference where tensor shapes vary widely across requests. This paper proposes a *Dynamically Reconfigurable Systolic Array* (DRSA); a single-mode-bit 16×16 architecture that switches at run time between one full 16×16 General Matrix Multiplication (GEMM) and four independent 8×8 GEMMs executing in parallel. Quadrant isolation is achieved by zeroing inter-boundary data wires; idle processing elements (PEs) are suppressed via a synchronous accumulator-clear signal, reducing dynamic power without lengthening the critical path. A parameterisable address generation unit (AGU) with dual stagger moduli drives both configurations from a single hardware path. Functional correctness is verified in simulation against a software golden model across all modes, including back-to-back mode switches. Post-synthesis results on a Xilinx Artix-7 100T at 100 MHz show a $4\times$ improvement in GEMM throughput for 8×8 workloads (7.84 M GEMM/s versus 1.82 M GEMM/s), a 43.6% reduction in per-GEMM latency, and only 5.11% total power overhead relative to a fixed-dimension baseline, at 0.255 W total on-chip power.

Index Terms—Systolic Arrays (SAs), Matrix Multiplication, Reconfigurable Architecture, Parallel Processing

I. INTRODUCTION

SAs are parallel computing architectures consisting of regularly interconnected PEs organised in a two-dimensional grid. Their intrinsic parallelism and effective data reuse capabilities render them highly suitable for the acceleration of matrix multiplication— an essential operation in the fields of machine learning, signal processing, and scientific computing [1]–[3].

In matrix multiplication $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, each output element c_{ij} arises from the dot product of the i -th row of $\mathbf{A}$ and the j -th column of $\mathbf{B}$. A weight-streaming $M \times N$ SA leverages this pattern by propagating elements of $\mathbf{A}$ horizontally and elements of $\mathbf{B}$ vertically, so all engaged PEs can execute concurrent multiply-accumulate (MAC) operations. Accurate computation depends on operands reaching each PE at the same time, which is achieved through staggered read-address sequences generated by an AGU. Despite their efficiency, conventional SAs are designed for fixed dimensions and do not adapt to varying workloads. When matrices smaller than the array are processed, a significant fraction of PEs remain idle, wasting dynamic power. This under-utilisation is especially pronounced in modern LLM serving, where batch sizes,

sequence lengths, and tensor dimensions vary widely across inference requests [4], [5].

To address these limitations, this paper proposes a *DRSA*. A single run-time control bit partitions the 16×16 array into four independent 8×8 sub-arrays, enabling simultaneous execution of four smaller GEMMs. When the full 16×16 mode is selected, all quadrants cooperate as a single array. The main contributions are:

- A dynamically reconfigurable 16×16 SA supporting two run-time modes, implemented in synthesisable Verilog-2001.
- A mode-aware AGU with dual stagger moduli ($S=16$ for Mode 0; $S=8$ for Mode 1) driven by a single hardware path.
- Quadrant isolation via wire-zeroing at the column-8 and row-8 boundaries, enabling four independent 8×8 GEMMs—verified by simulation including back-to-back mode switches.
- Post-synthesis evaluation on Xilinx Artix-7 100T demonstrating $4\times$ GEMM-throughput at 5.11% total power overhead and 0.255 W absolute on-chip power.

The remainder of this paper is organised as follows: Section II reviews related work. Section III describes the proposed architecture. Section IV details the implementation. Section V presents and discusses experimental results. Section VI concludes the paper.

II. RELATED WORK

SAs were formalised by Kung and Leiserson [6] as architectures in which data flows rhythmically through a mesh of tightly coupled PEs, enabling high throughput at low memory bandwidth. Kung later argued that systolic regularity makes them uniquely amenable to Very Large Scale Integration (VLSI) implementation [7]. These insights underpin the Google Tensor Processing Unit (TPU), which employs a 256×256 weight-stationary SA for deep-learning inference [8], [9]. Recent work has applied systolic principles to LLM hardware, including MLA-aware group vector systolic accelerators [4].

Early Field-Programmable Gate Array (FPGA) implementations demonstrated that reconfigurable logic could match ASIC-grade data-path regularity while retaining post-fabrication flexibility [10]. Subsequent designs exploited DSP48 slices for inner-product kernels and block RAMs for operand buffering [11].

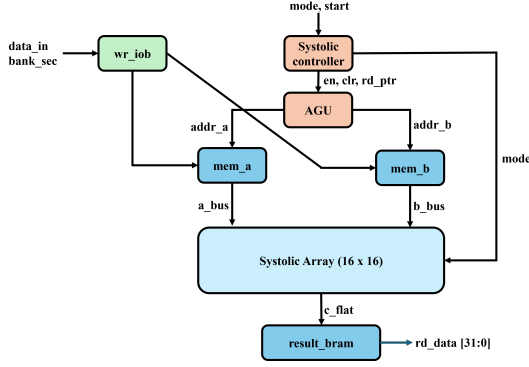


Fig. 1. Top-level block diagram of the proposed DRSA.

Reconfigurability has been explored at two granularities. *Coarse-grained* approaches partition a large array at compile time, routing distinct workloads to disjoint PE clusters [12]. *Fine-grained* approaches insert run-time multiplexers at PE boundaries, enabling switching between a single large GEMM and several smaller GEMMs within a clock domain [3], [13]. Accumulator clearing to suppress switching activity in idle PEs has been shown to reduce dynamic power without affecting functional correctness [14]. Parameterisable AGU designs supporting variable stagger moduli allow the same hardware to serve both square and rectangular operand tiles [15].

None of the above works simultaneously demonstrate run-time mode switching between a full-array GEMM and a partitioned multi-GEMM, a serial I/O-efficient load interface, and simulation-verified functional correctness on a resource-constrained Artix-7 device within a single unified architecture. The present work addresses this gap.

III. PROPOSED ARCHITECTURE

A. System Overview

Fig. 1 shows the top-level block diagram of the proposed DRSA. The design performs integer matrix multiplication $C = A \times B$ with $A, B \in \mathbb{Z}^{16 \times 16}$, 8-bit unsigned elements and 32-bit accumulated outputs. A single mode bit selects between:

- **Mode 0**—one full 16×16 GEMM (dot-product depth 16, $K_{\text{eff}}=31$, latency 55 cycles) as shown in Fig. 3
- **Mode 1**—four independent 8×8 GEMMs in parallel across the four quadrants (dot-product depth 8, $K_{\text{eff}}=14$, latency 31 cycles) as shown in Fig. 3

The top-level hierarchy comprises five sub-modules: a systolic controller (ctrl_16x16), an AGU (agu_16x16), two banked operand memories (mem_bank_16x16 for A and B), a streaming PE array (array_16x16), and a result buffer (result_bram_16x16).

B. Processing Element

Each PE (pe_16x16) implements a two-stage pipelined MAC cell. Stage 1 registers the product:

$$\text{prod_reg} \leftarrow a_in \times b_in, \quad (1)$$

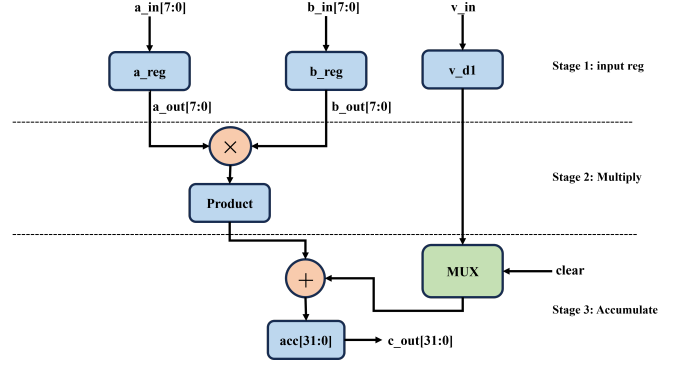


Fig. 2. Internal data stream of a singular PE. Stage 1 registers the 8-bit inputs; Stage 2 evaluates the 16-bit product; Stage 3 conditionally aggregates into the 32-bit output register. The clear command resets the accumulator via the MUX. The blue arrow indicates the feedback path of the accumulator.

and Stage 2 conditionally accumulates:

$$\text{acc} \leftarrow \text{acc} + \text{prod_reg} \quad (\text{when } v_in_d1 \text{ asserted}). \quad (2)$$

The PE also forwards a_in to the right neighbour and b_in to the lower neighbour. A synchronous `clear` input resets the accumulator without an asynchronous path, simplifying timing closure. Fig. 2 shows the internal datapath of a single PE.

C. Operand Memory and Serial Load Interface

Operand matrices are stored in 32 single-port BRAM18 banks of depth 32, each 8 bits wide. All 32 banks share data and address buses; each bank's write-enable is independently asserted via a one-hot decoder. Writing proceeds serially: one byte per cycle, with a 5-bit `bank_sel`, a 5-bit `wr_addr`, a `mem_sel` flag (selecting matrix A or B), and an 8-bit `data_in`.

D. Address Generation Unit

AGU generates staggered read-address sequences that align operand wavefronts at the PE grid boundary. For each bank $k \in \{0, \dots, 31\}$, the BRAM depth is 32 and `rd_ptr` sweeps $[0, K_{\text{eff}}]$. In Mode 0, $K_{\text{eff}}=31$; in Mode 1, $K_{\text{eff}}=14$ (with $S=8$ and $N_{\text{sub}}=8$, giving $K_{\text{eff}} = S-1 + N_{\text{sub}}-1 = 14$). The address for bank k is:

$$\text{addr}[k] = \begin{cases} \text{rd_ptr} - k & \text{Mode 0} \\ \text{rd_ptr} - (k \bmod S) & \text{Mode 1} \end{cases} \quad (3)$$

subject to `valid_mask[k]` gating the output to zero until the wavefront arrives at bank k .

E. Mode-1 Partitioning and Isolation

In Mode 1 the array is divided into four 8×8 quadrants by zeroing inter-quadrant wires at the column-8 and row-8 boundaries:

$$h_w[i][8] = 0 \quad \forall i \in [0, 7], \quad (4)$$

$$v_w[8][j] = 0 \quad \forall j \in [0, 7]. \quad (5)$$

The valid-token bus `v_in_rows` is mirrored so that the lower eight rows receive the same stagger sequence as the upper

eight rows, and the AGU uses $S=8$ so all four sub-arrays begin their wavefronts simultaneously. Fig. ?? illustrates both modes.

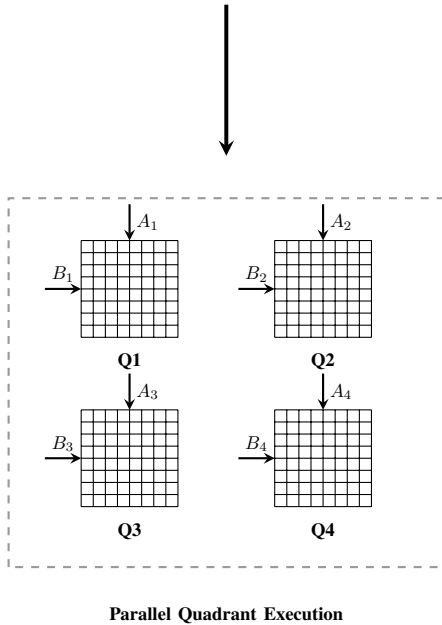
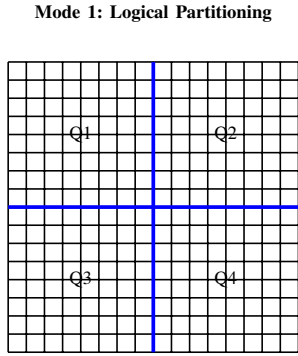
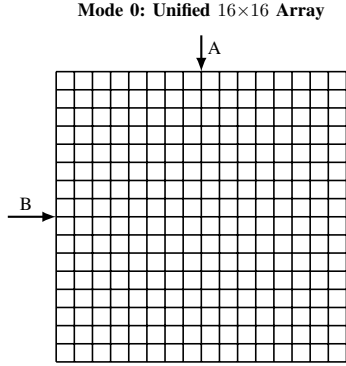


Fig. 3. Full 16x16 operating modes. Mode 0 (top): all 256 PEs cooperate one unified array for a 16x16 GEMM. Mode 1 (bottom): blue lines mark zero/wire isolation boundaries that partition the array into four independent 8x8 quadrants (Q1-Q4) each executing separate GEMM simultaneously.

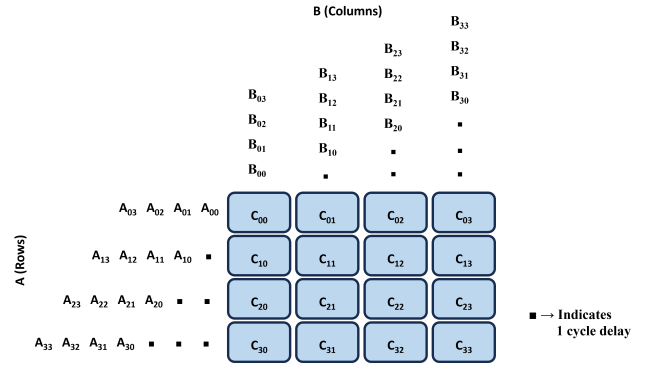


Fig. 4. Data stagger pattern illustrated on a represented 4x4 array. Row i of matrix A is injected at the left edge with a delay of i cycles; column j of matrix B is injected at the top edge with a delay of j cycles.

F. Systolic Controller

The controller drives a three-phase FSM: (i) a *clear phase* of $C_{\text{clr}}=4$ cycles that asserts `clr` to flush PE accumulators; (ii) a *compute phase* in which `en` is asserted and `rd_ptr` increments from 0 to K_{eff} ; and (iii) a *drain phase* of D cycles to flush the pipeline before asserting `done`. Mode-dependent parameters are latched at the rising edge of `start`:

Mode 0: $K_{\text{eff}}=31$, $D=20 \Rightarrow$ done at cycle 55

Mode 1: $K_{\text{eff}}=14$, $D=13 \Rightarrow$ done at cycle 31

IV. IMPLEMENTATION

A. Design Tools and Target Device

The entire DRSA was described in synthesizable Verilog-2001 and implemented using **Xilinx Vivado 2025.1** targeting the **Artix-7 100T** device (xa7a100tfgg484-2I). The design was synthesized under explicit timing and resource constraints, with a target clock period of 10 ns (100 MHz). A constrained synthesis strategy was employed to ensure timing closure and optimal resource utilization.

Functional correctness was verified using a Verilog testbench that computes a software golden reference and compares all 256 output elements for each operating mode, including back-to-back mode switches at cycle boundaries. Post-synthesis power estimation was performed using a Switching Activity Interchange Format (SAIF) file generated from the testbench simulation in Vivado.

B. Resource Utilisation

Table I summarises post-synthesis logic utilisation. The baseline is a fixed 16x16 design operating in Mode 0 only, with no mode-switching logic.

The 49.3% register increase is the primary overhead of Mode 1, arising from the mirrored valid-token bus, mode-aware AGU stagger registers, and the registered result-MUX path. BRAM and DSP tile counts are unchanged. The 3,264 F7/F8 muxes implement the mode-aware wire-zeroing logic. No DSP slices are inferred; multipliers map to Look-Up Table (LUT)-carry chains, appropriate for 8-bit operands on Artix-7.

TABLE I
POST-SYNTHESIS LOGIC UTILISATION (ARTIX-7 100T)

Resource	Base	DRSA	Avail.	Δ
Slice LUTs	34,220	36,303	63,400	+6.08%
Slice Registers	16,591	24,772	126,800	+49.3%
F7/F8 Muxes	0	3,264	47,550	—
BRAM Tiles	32	32	135	0.00%
DSP48E1	0	0	240	0.00%

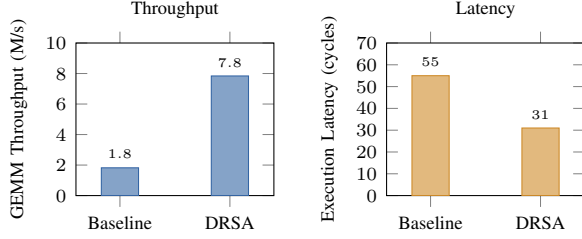


Fig. 5. GEMM throughput (left) and execution latency in cycles (right) for the baseline vs. the DRSA (Mode 1) at 100 MHz.

C. Timing Closure

The design meets timing at 100 MHz. The critical path traverses the 8-bit unsigned multiplier in the PE, the 32-bit accumulator adder, and two levels of MUX logic in the AGU.

V. RESULTS AND DISCUSSION

A. Throughput and Latency

Table II compares execution latency and GEMM throughput at 100 MHz for the DRSA versus the Mode-0-only baseline. Latency figures are derived directly from the controller FSM timing described in Section III.

TABLE II
LATENCY AND THROUGHPUT AT 100 MHz VS. BASELINE

Metric	Base	DRSA	Change.
Latency (cycles)	55	31	-43.6%
Throughput (M GEMM/s)	1.82	7.84	+436%
Compute (GOPS)	0.942	0.942	0%

The $4.3\times$ GEMM throughput gain arises from both the shorter per-GEMM latency (31 vs. 55 cycles) and the four-way parallelism. Raw compute throughput is identical in both modes since the same 256 PEs remain active throughout. Fig. 5 visualises the throughput and latency comparison against the baseline.

B. Power Analysis

Post-synthesis power estimation was performed in Vivado using an SAIF activity file from simulation (confidence: Medium). Table III presents the results compared against the baseline.

The 5.11% total power overhead confirms that the Mode 1 isolation logic introduces negligible dynamic switching cost. Static power is unchanged at 0.099 W. Clock distribution dominates dynamic power at 0.078 W, consistent with the 24,836 clock fanout seen in the synthesis report. Fig. 6 visualises the power breakdown.

TABLE III
POST-SYNTHESIS POWER (ARTIX-7 100T, 100 MHz, 25°C)

Component	Base (W)	DRSA (W)
Clocks	0.069	0.078
Slice Logic	0.007	0.008
Signals	0.004	0.005
Block RAM	0.063	0.065
I/O	<0.001	<0.001
Dynamic Total	0.149	0.157
Device Static	0.099	0.099
Total On-Chip	0.243	0.255
Power overhead	+5.11%	

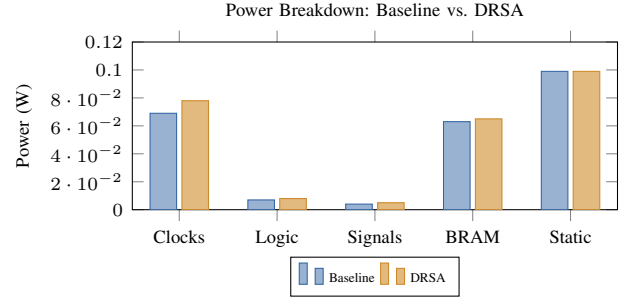


Fig. 6. Per-component power breakdown for the baseline and DRSA designs. Static power is identical; the small dynamic overhead is dominated by increased clock-tree activity.

C. Summary of Improvements over Baseline

Table IV consolidates all key metrics comparing the DRSA against the fixed Mode-0 baseline. The DRSA achieves a $4.3\times$ improvement in GEMM throughput for 8×8 workloads at a cost of only 6.08% additional LUTs and 5.11% additional total power.

TABLE IV
DRSA VS. BASELINE: SUMMARY OF KEY METRICS

Metric	Base	DRSA	Δ
Throughput (M GEMM/s)	1.82	7.84	+436%
Latency (cycles)	55	31	-43.6%
Compute (GOPS)	0.942	0.942	0%
Slice LUTs	34,220	36,303	+6.08%
Slice Registers	16,591	24,772	+49.3%
BRAM Tiles	32	32	0%
DSP48E1	0	0	0%
Total Power (W)	0.243	0.255	+5.11%

VI. CONCLUSION

This paper presented a 16×16 DRSA supporting two run-time modes via a single control bit. Post-synthesis results on a Xilinx Artix-7 100T at 100 MHz demonstrate a favourable architectural trade-off: a 6.08% LUT overhead and 5.11% power overhead yield a $4.3\times$ improvement in GEMM throughput for 8×8 workloads (7.84 M/s vs. 1.82 M/s), while sustaining 0.942 GOPS and consuming only 0.255 W total on-chip power. BRAM and DSP utilisation are unchanged. Functional correctness is verified by simulation across all modes, including back-to-back switches. Future work will extend the approach

to support three or more partition levels and target higher-density UltraScale+ devices for transformer attention kernels and batched LLM decoding.

REFERENCES

- [1] A. Puşcaşu, C. B. Ciobanu, and O. Buiu, “Systolic array matrix multiplication accelerator,” in *2024 International Semiconductor Conference (CAS)*, 2024, pp. 207–210.
- [2] H. Yuan, J. Bi, Z. Wang, J. Zhang, M. Zhou, and R. Buyya, “Multi-perspective and energy-efficient deep learning in edge computing,” *IEEE Internet of Things Journal*, vol. PP, pp. 1–1, 01 2025.
- [3] F. Yang, N. Li, L. Wang, P. Jiang, X. S. Miao, and X. Wang, “Isara: An island-style systolic array reconfigurable accelerator based on memristors for deep neural networks,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. PP, pp. 1–13, 01 2025.
- [4] S. Huang, T. Wang, K. Jiang, A. Shen, K. Li, A. Li, M. Huang, and H. Yu, “An mla-llm hardware acceleration with tensor-train decomposition on group vector systolic accelerator,” in *2025 IEEE 36th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2025, pp. 101–108.
- [5] T. Zhao, S. Lu, C. Wu, and L. He, “Chatopu: An fpga-based overlay processor for large language models with unstructured sparsity,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD ’24. New York, NY, USA: Association for Computing Machinery, 2025.
- [6] H. T. Kung, C. E. Leiserson *et al.*, “Systolic arrays (for vlsi),” in *Sparse Matrix Proceedings 1978*, vol. 1. SIAM Philadelphia, PA, USA, 1979, pp. 256–282.
- [7] H.-T. Kung *et al.*, “Why systolic architecture?” 1982.
- [8] F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. Lu, M.-F. Chang, D. Ielmini, Y. Yang *et al.*, “Hardware implementation of memristor-based artificial neural networks,” *Nature communications*, vol. 15, no. 1, p. 1974, 2024.
- [9] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [10] P. Babu and P. Eswaran, “Reconfigurable fpga architectures: A survey and applications,” *Journal of The Institution of Engineers (India) Series B*, vol. 102, p. 143–156, 11 2020.
- [11] N. Suda, V. Chandra, G. Dasika, A. Mohanty, Y. Ma, S. Vrudhula, J.-s. Seo, and Y. Cao, “Throughput-optimized opencl-based fpga accelerator for large-scale convolutional neural networks,” in *Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays*, 2016, pp. 16–25.
- [12] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” *ACM SIGARCH computer architecture news*, vol. 44, no. 3, pp. 367–379, 2016.
- [13] H. Kwon, A. Samajdar, and T. Krishna, “Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects,” *ACM Sigplan Notices*, vol. 53, no. 2, pp. 461–475, 2018.
- [14] K. Krishnan, P. D. S. K., and S. K., “Power efficient systolic array design for machine learning acceleration,” 07 2025, pp. 1–4.
- [15] K. Huang, X. Zhang, D. Zheng, M. Yu, X. Jiang, X. Yan, L. Brisolara, and A. Jerrya, “A scalable and adaptable ilp-based approach for task mapping on mpsoe considering load balance and communication optimization,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. PP, pp. 1–1, 07 2018.